Table of Contents

Abstract 3

1. Introduction 4

2.1 Dictionary for Books (ISBN → Book Details) 4

Why Dictionary was Chosen: 4

Alternative Considered: 4

2.2 List for Members 4

Why List was Chosen: 4

Alternative Considered: 5

2.3 Tuple for Valid Genres 5

Why Tuple was Chosen: 5

Alternative Considered: 5

3. System Architecture Design 5

3.1 Modular Function-Based Design 5

Approach: 5

Key Design Patterns: 6

3.2 Core Operations Implementation 6

CRUD Operations Design: 6

Borrowing System Constraints: 6

- Availability Checking: 6

- **Return Validation:** 6

4. Error Handling and Validation Strategy 6

4.1 Comprehensive Validation 6

Input Validation: 6

Business Rule Enforcement: 7

4.2 User-Friendly Error Messages.....7

5. Testing Strategy Rationale.....7

5.1 Test Coverage Approach 7

The testing strategy covers: 7

5.2 Assert-Based Testing 7

Using Python's `assert` statements provides: 7

Screen Shoot of the code.....8

8. Conclusion 17

9. Reference 18

Abstract

This paper presents the design rationale for a Mini Library Management System implemented in Python, focusing on the strategic selection of core data structures—dictionary, list, and tuple—to optimize performance, maintainability, and functional requirements. The dictionary data structure was employed for book storage using ISBN as unique keys, providing O(1) time complexity for essential operations such as retrieval, updates, and existence checks. Members were managed using a list structure, offering adequate O(n) performance for expected system scales while maintaining implementation simplicity. Genre validation was implemented through an immutable tuple, ensuring data integrity and consistent categorization across the system. The design demonstrates how careful data structure selection directly supports CRUD operations, borrowing constraints, and search functionality while providing clear pathways for future scalability. This approach balances computational efficiency with code maintainability, creating a robust foundation that aligns with both academic principles and practical library management needs.

1. Introduction

This document explains the design decisions and architectural choices made in developing the Mini Library Management System. The system implements core library operations using Python's fundamental data structures while adhering to software engineering principles.

2. Data Structure Selection Rationale

2.1 Dictionary for Books (ISBN → Book Details)

Why Dictionary was Chosen:

- O(1) Lookup Time: The primary operation in a library system is searching for books by their unique identifier (ISBN). Dictionaries provide constant-time complexity for lookups, making them ideal for frequent search operations.

- Natural Key-Value Mapping: The ISBN serves as a perfect natural key that uniquely identifies each book, while the book details (title, author, genre, copies) form the corresponding value.

- Duplicate Prevention: Dictionary keys are inherently unique, automatically preventing duplicate ISBN entries without additional validation code.

- Efficient CRUD Operations: All Create, Read, Update, Delete operations can be performed efficiently with minimal computational overhead.

Alternative Considered:

Using a list would require O(n) linear search for every operation, which would be inefficient as the library grows.

2.2 List for Members

Why List was Chosen:

- Ordered Collection: Members benefit from maintaining insertion order, which can be useful for displaying members in the order they joined.

- Simple Iteration: Lists provide straightforward iteration for operations like displaying all members or generating reports.

- Appropriate Scale: For a mini library system, the number of members is typically manageable with linear operations.

- Dynamic Sizing: Lists automatically resize as members are added or removed.

Alternative Considered:

A dictionary keyed by member ID would provide faster lookups but was deemed unnecessary given the expected scale and the benefit of maintaining order.

2.3 Tuple for Valid Genres

Why Tuple was Chosen:

- **Immutability:** Genres represent a fixed set of categories that shouldn't change during program execution. Tuples enforce this immutability.

-**Data Integrity:** Prevents accidental modification of valid genres at runtime, ensuring consistency across the system.

- **Memory Efficiency:** Tuples are more memory-efficient than lists for fixed data sets.

Alternative Considered:

A list would allow runtime modifications but could lead to inconsistent genre validation.

3. System Architecture Design

3.1 Modular Function-Based Design

Approach:

The system uses a modular function-based architecture rather than object-oriented design to:

- Maintain simplicity for the assignment scope

- Provide clear, testable units of functionality

- Allow easy understanding for beginners

- Enable straightforward unit testing

Key Design Patterns:

- **Separation of Concerns:** Each function handles one specific responsibility

- **Input Validation:** All functions validate inputs before processing

- Error Handling: Comprehensive error checking with meaningful messages

- **Stateless Operations:** Functions operate on global data structures for simplicity

3.2 Core Operations Implementation

CRUD Operations Design:

- Create: Validation of uniqueness and data integrity before addition

- Read: Efficient search with multiple access patterns (ISBN, title, author)

- **Update:** Constraint checking before modification (e.g., genre validation)

- **Delete:** Dependency checking to maintain data consistency

Borrowing System Constraints:

- Availability Checking: Ensures books are available before borrowing

- Member Limits: Enforces maximum 3-book borrowing limit

- **Duplicate Prevention:** Prevents same member from borrowing multiple copies

- **Return Validation:** Ensures only borrowed books can be returned

4. Error Handling and Validation Strategy

4.1 Comprehensive Validation

Input Validation:

- ISBN uniqueness checking

- Genre validity against predefined tuple

- Non-negative copy counts

- Member ID uniqueness

- Email format validation (implicit through structure)

Business Rule Enforcement:

- Borrowing limits (max 3 books per member)

- Availability constraints (cannot borrow unavailable books)

- Deletion dependencies (cannot delete borrowed books or members with books)

4.2 User-Friendly Error Messages

Each validation failure provides specific, actionable error messages to guide users toward correct system usage.

5. Testing Strategy Rationale

5.1 Test Coverage Approach

The testing strategy covers:

- Normal Operations: Valid scenarios to ensure basic functionality

- Edge Cases: Boundary conditions and limits

- Error Conditions: Invalid inputs and constraint violations

- **Integration Scenarios:** Multi-step operations and dependencies

5.2 Assert-Based Testing

Using Python's `assert` statements provides:

- Simple, readable test cases

- Automatic test failure detection

- Minimal external dependencies

- Easy integration with development workflow

README.md

 # Mini Library Management System (Python)

Overview

This project implements a simple library management system in Python. It demonstrates CRUD operations using basic data structures and functions.

Features

- Add, search, update, delete books

- Add, search, delete members

- Borrow and return books

- Data stored in lists and dictionaries

Files

- `operations.py` - core logic

- `demo.py` - menu-driven interface

- `test_operation.py` - test cases

- `UML_diagram.png` - structure diagram

How to Run

1. Run `demo.py` for interactive menu.

2. Run `test_operation.py` to test functionality.

3. Run `operations.py` to check module message.

Data Structures

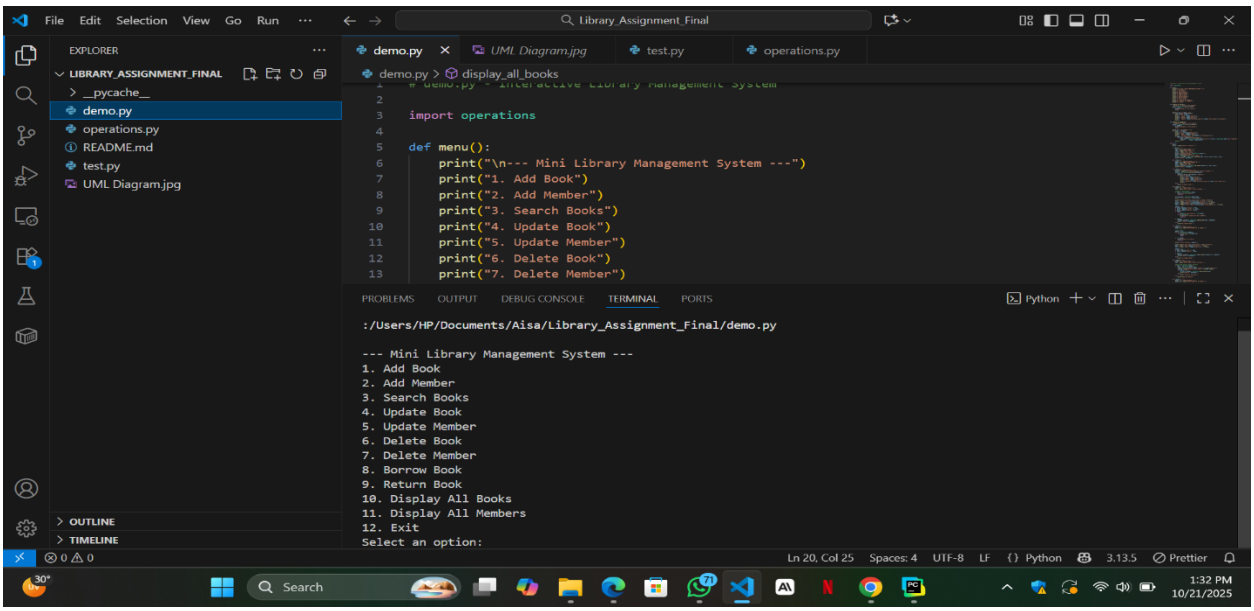
- Dictionary: `books`

- List of Dictionaries: `members`

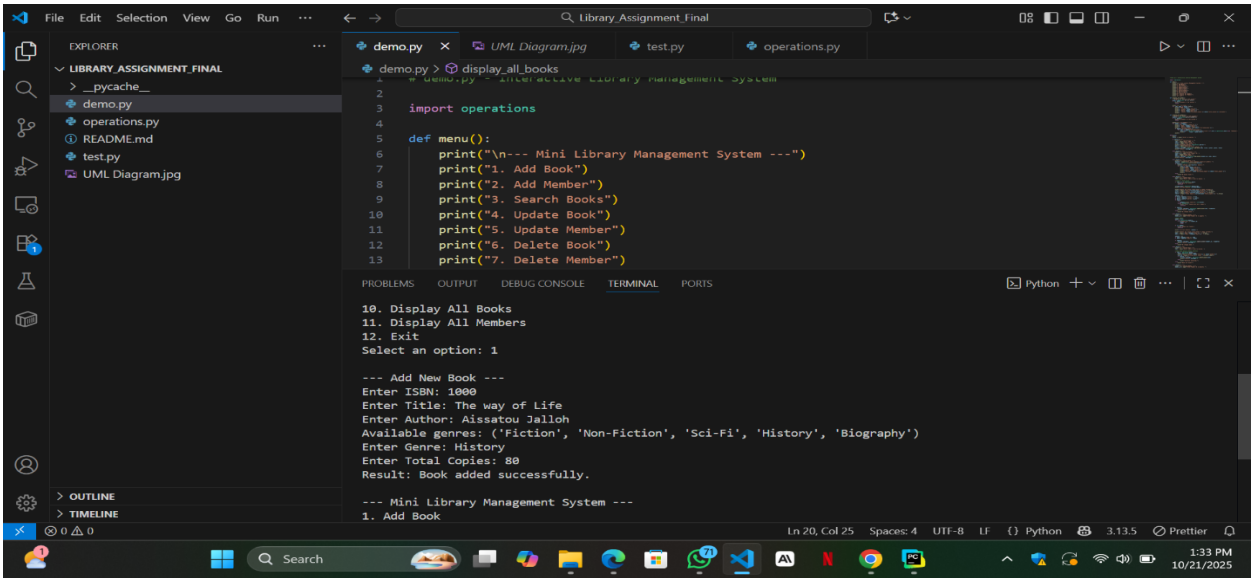
- Tuple: `genres`

Screen Shoot of the code

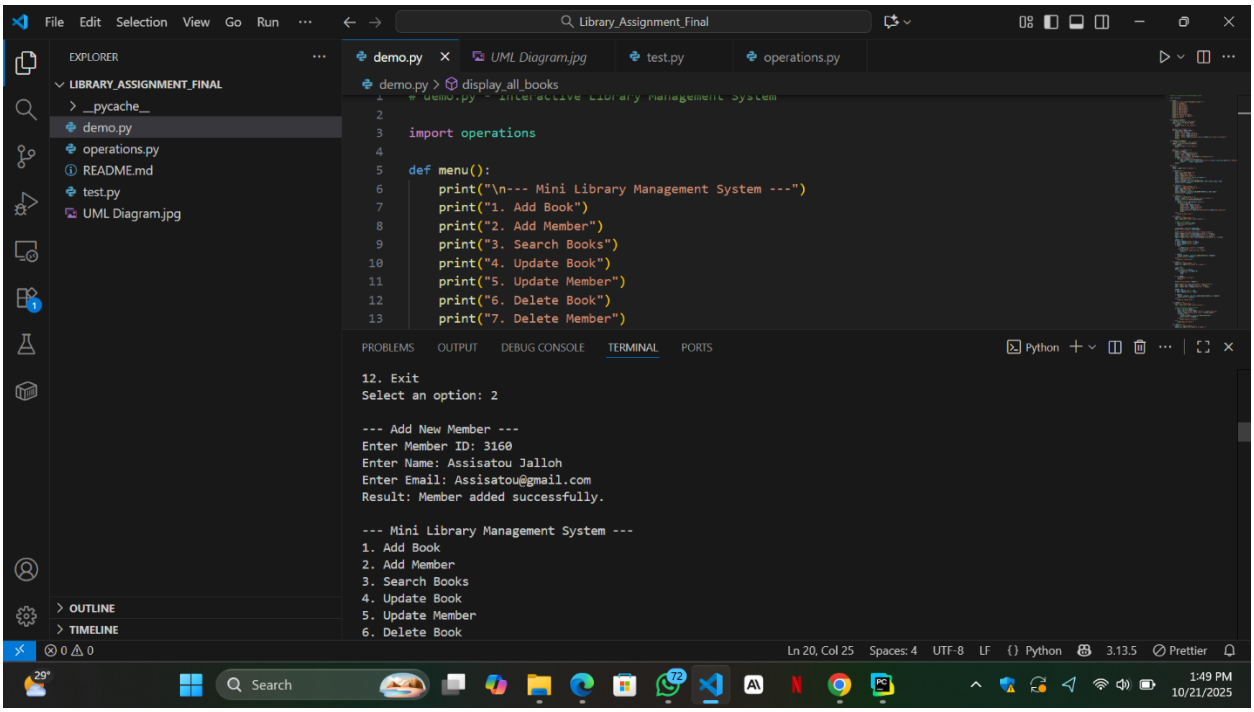
Demo.py Core Function



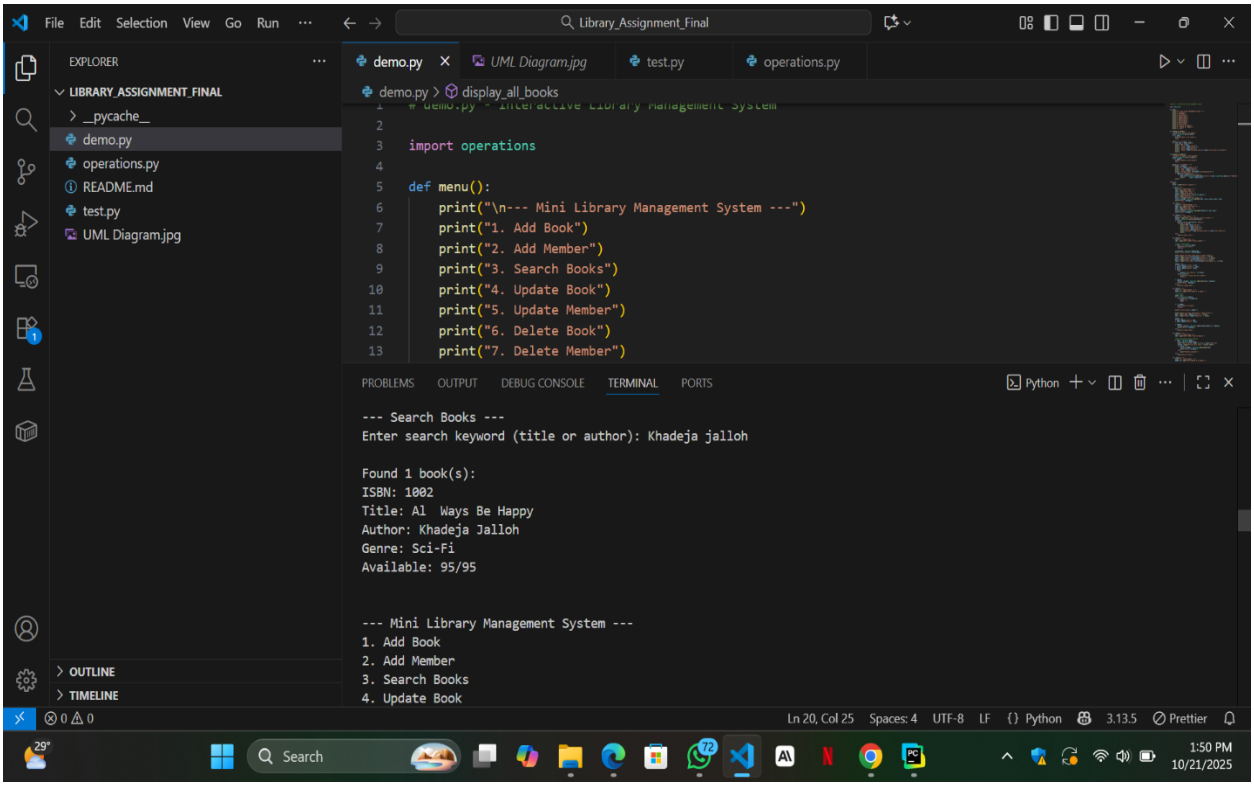
Add Book



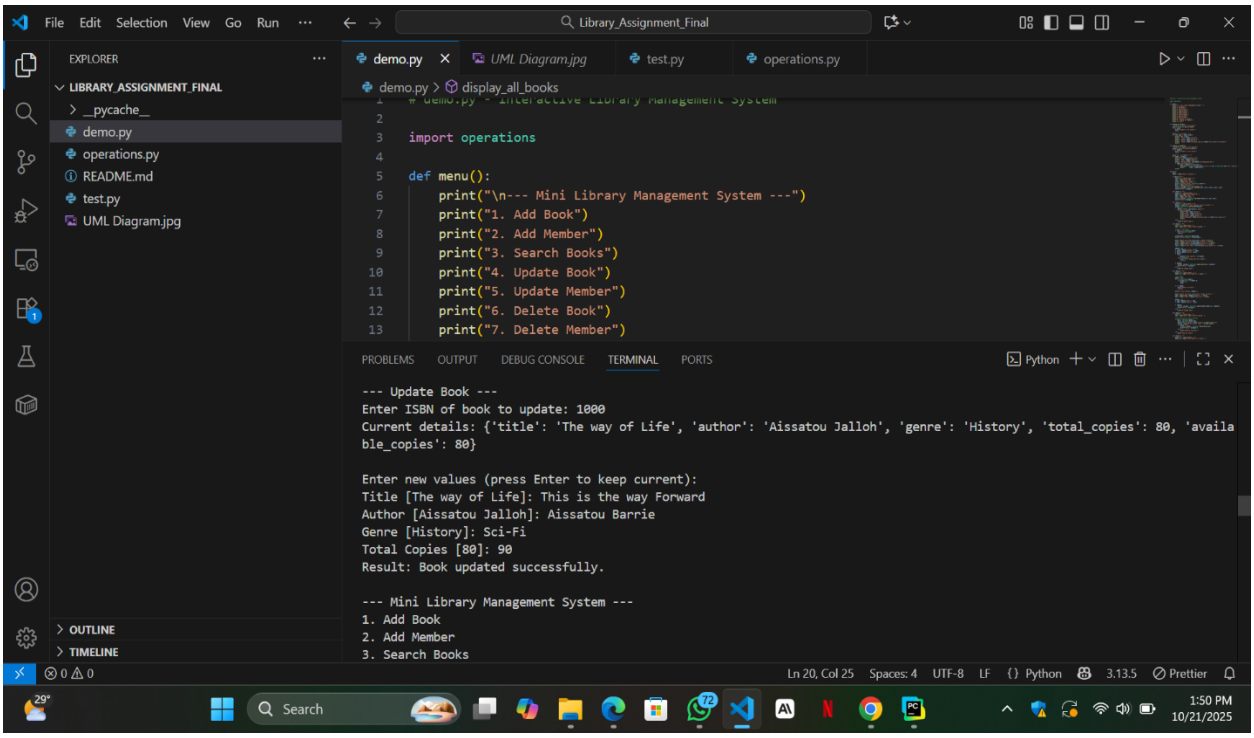
Add Member



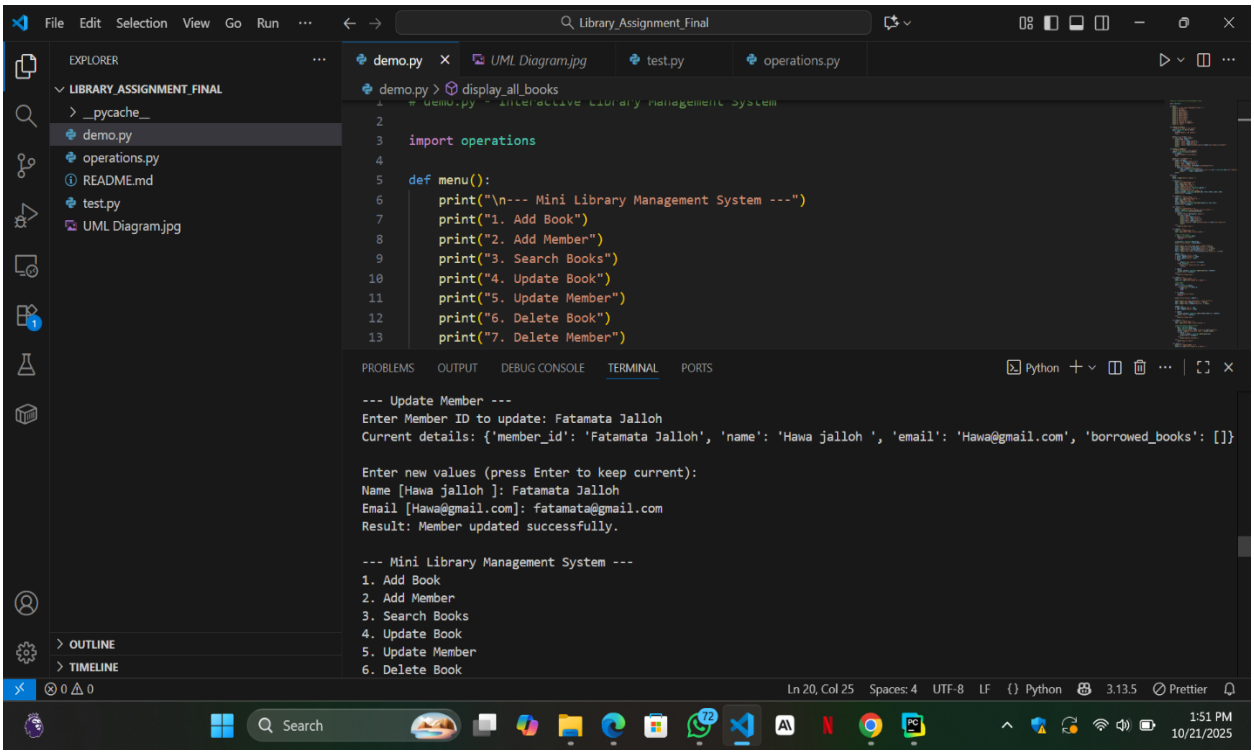
Search Book



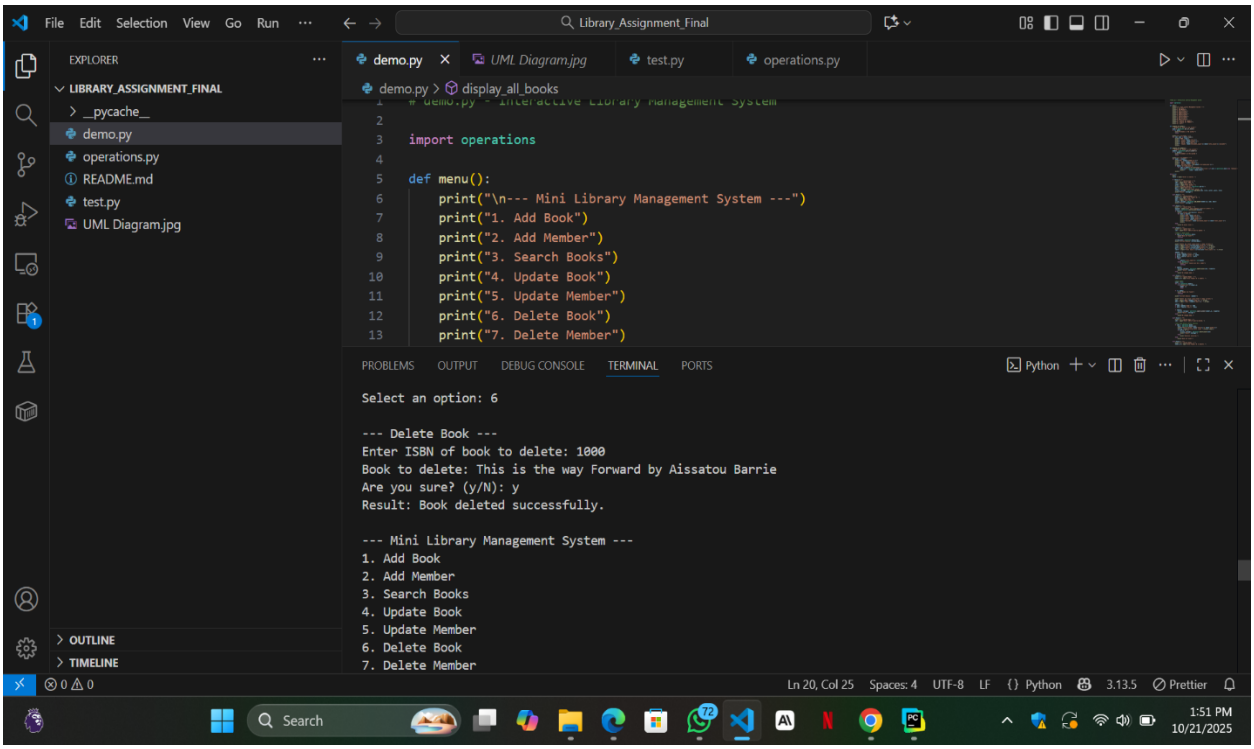
Update Book



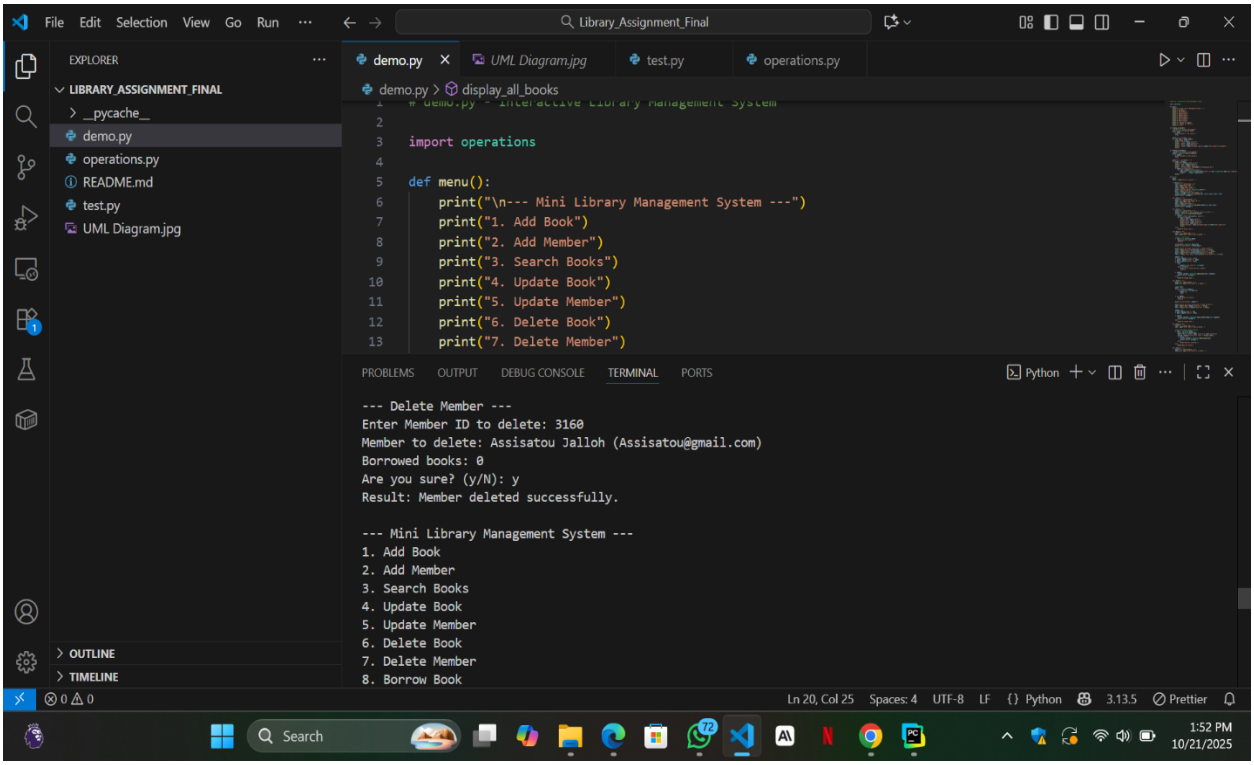
Update Member



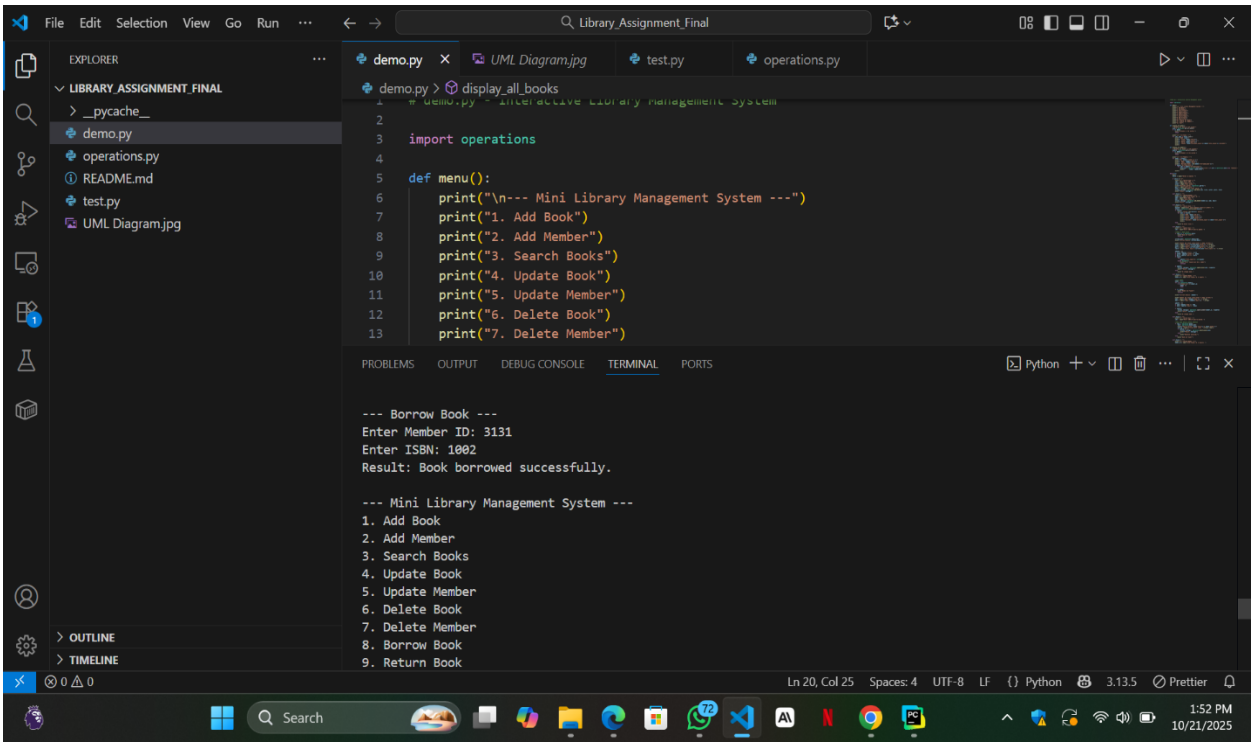
Delete Book



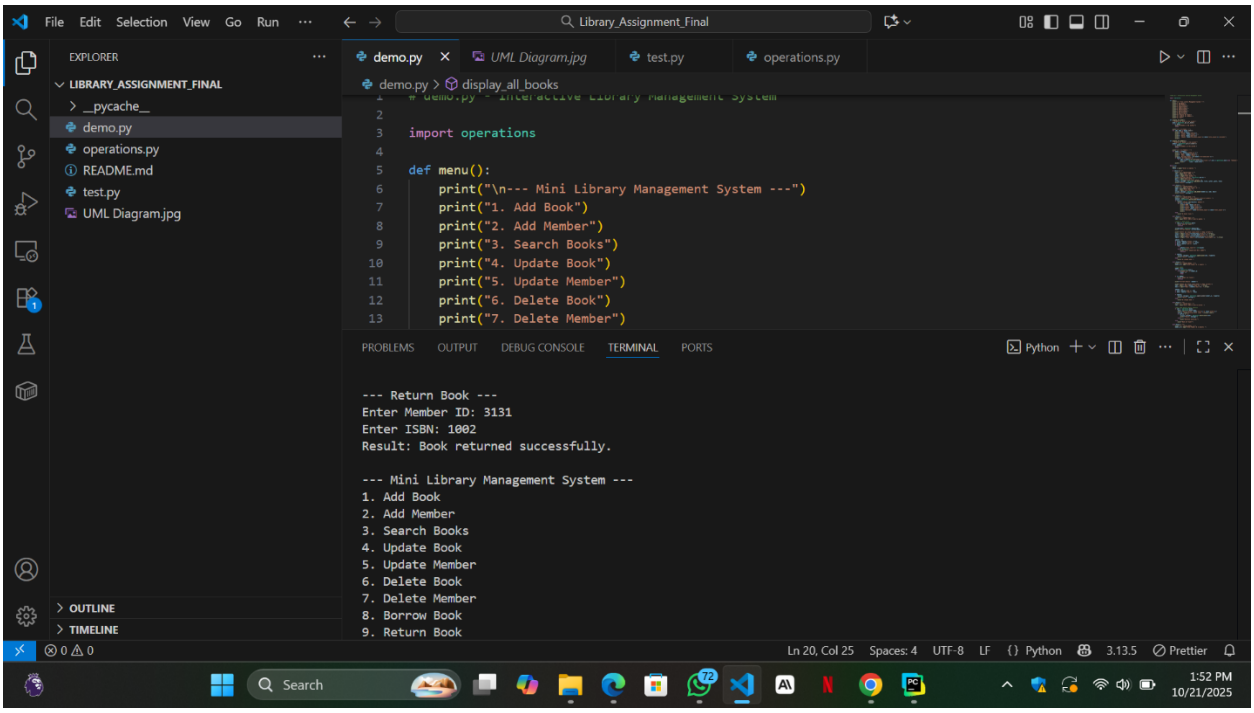
Delete Member



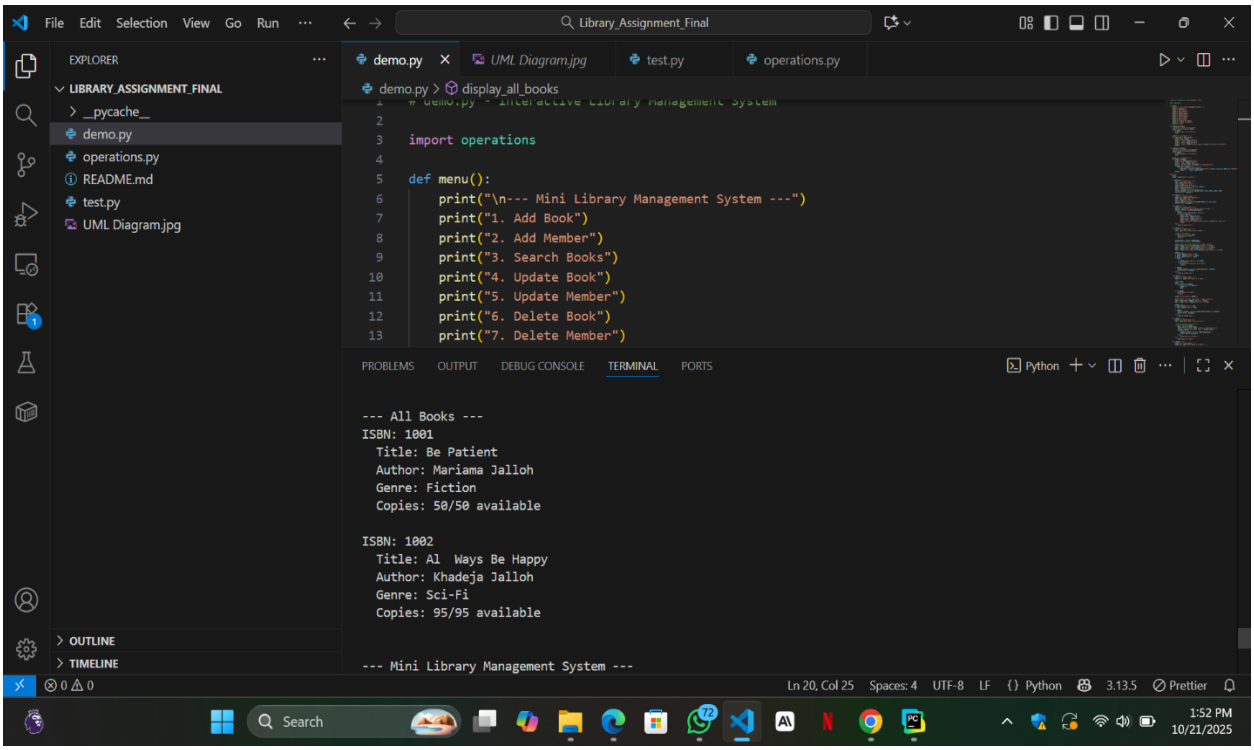
Borrow Book



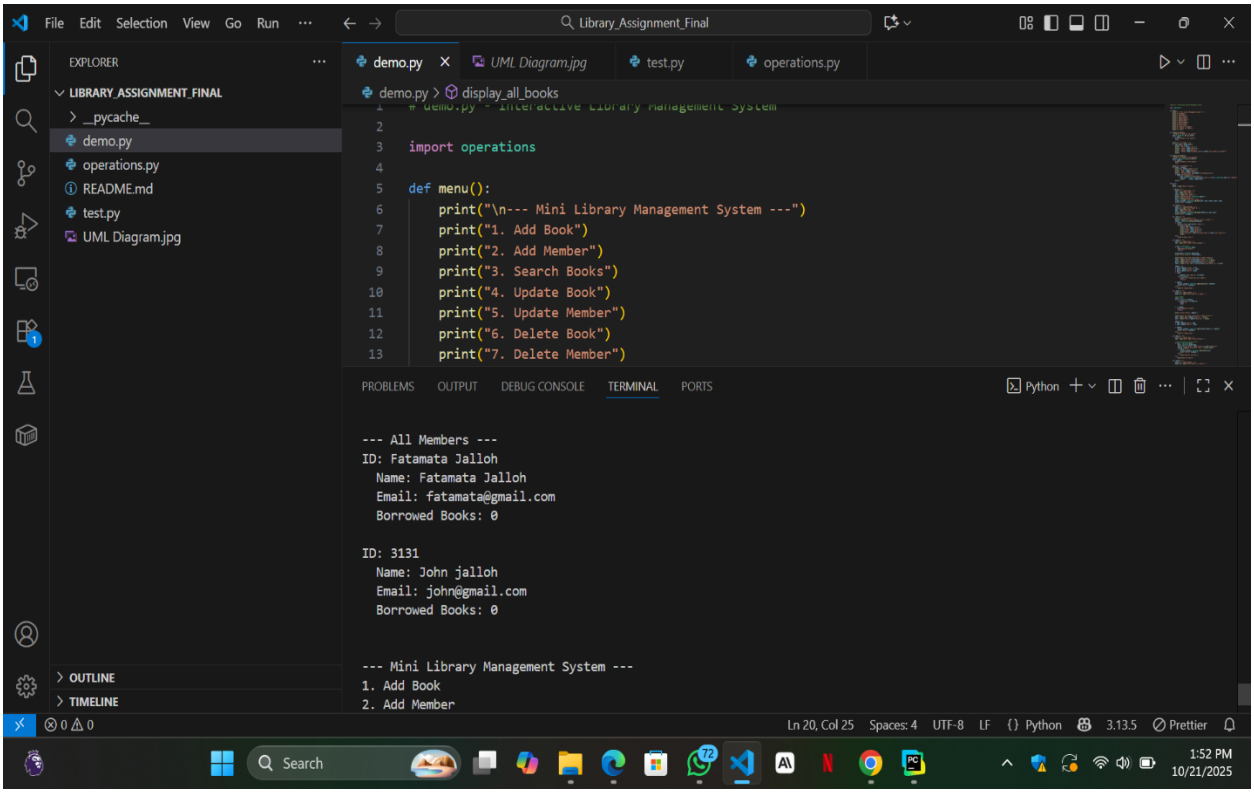
Return Book



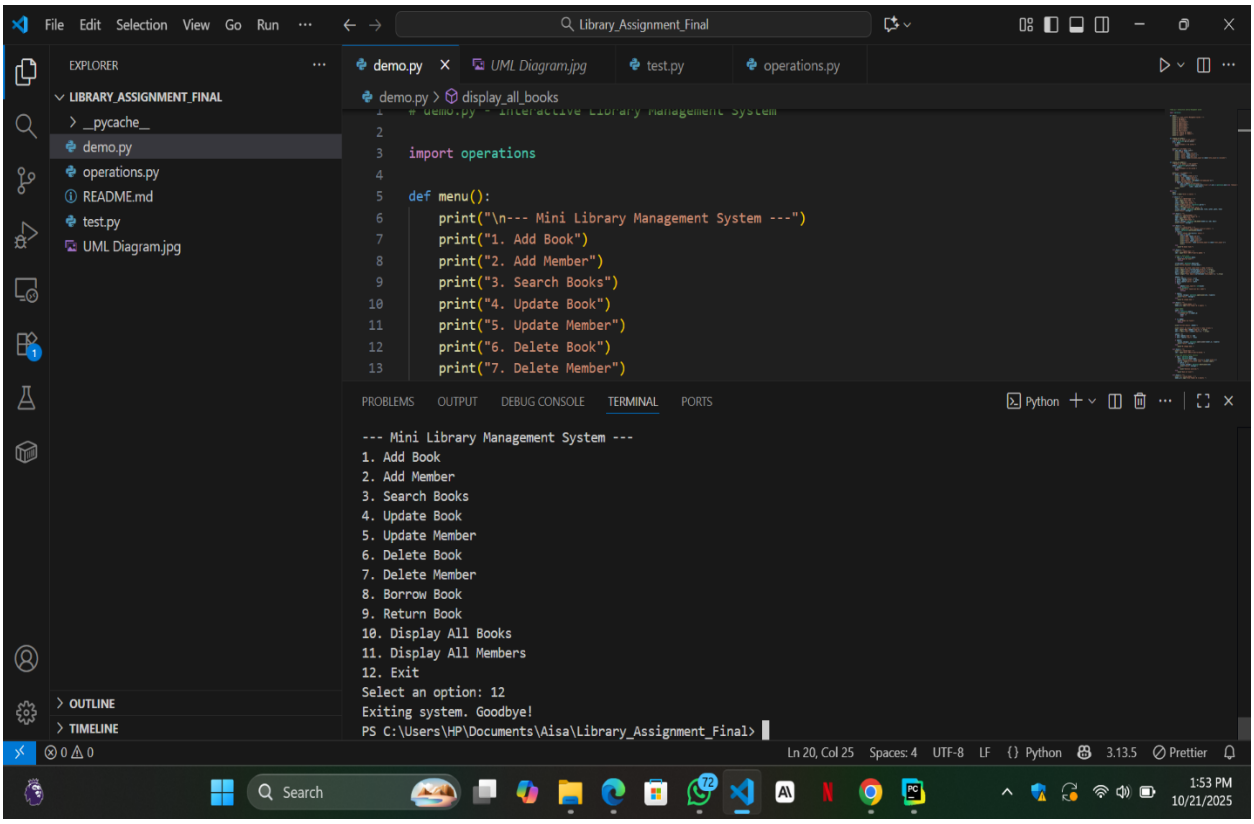
Displayed All Book



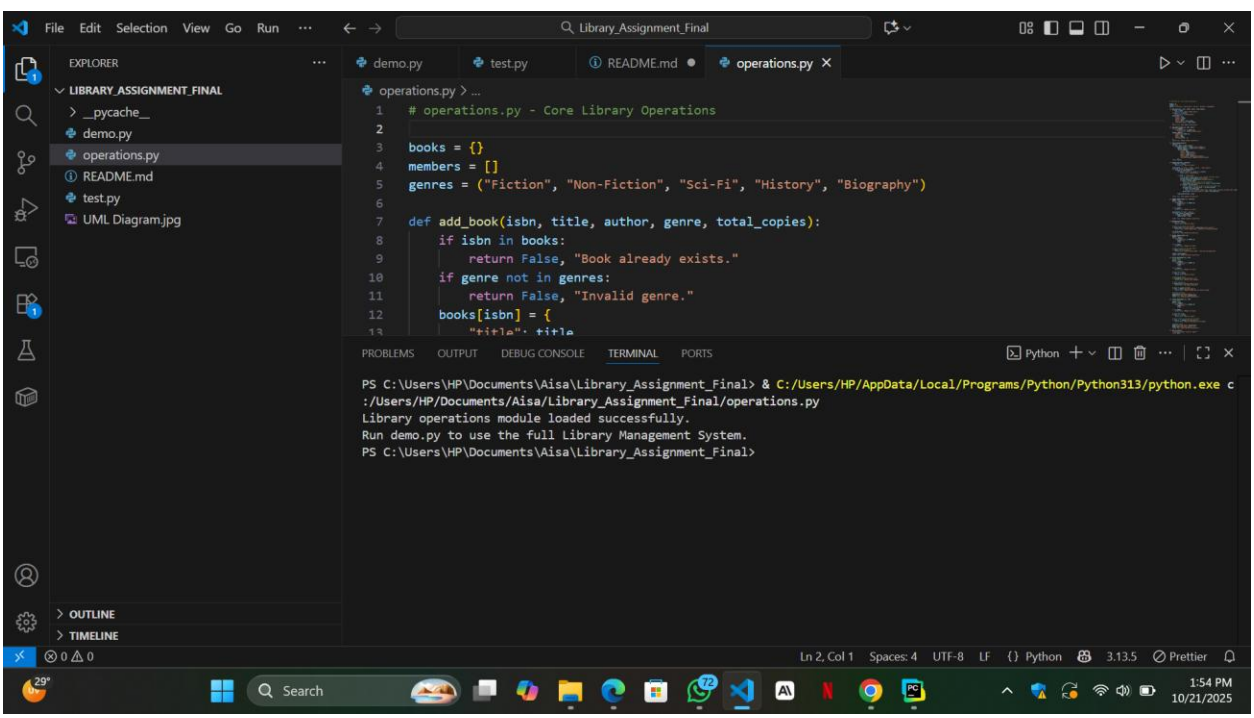
Displayed All Member



Exit



Operation.py



Test.py

```python
def run_tests():
    operations.borrow_book("M001", "007")

    # Try to borrow 4th book
    success, message = operations.borrow_book("M001", "008")
    assert success == False
    assert "limit" in message
    print("✅ borrow_book() - Borrow limit test passed")

    print("\n" + "=" * 50)
    print("🎉 ALL TESTS PASSED SUCCESSFULLY!")
    print("=" * 50)
```

```
8. Testing delete_book()...
✅ delete_book() - Basic delete test passed
✅ delete_book() - Borrowed book delete test passed

9. Testing delete_member()...
✅ delete_member() - Basic delete test passed
✅ delete_member() - Member with books delete test passed

10. Testing borrow limit...
✅ borrow_book() - Borrow limit test passed

==================================================
🎉 ALL TESTS PASSED SUCCESSFULLY!
==================================================
PS C:\Users\HP\Documents\Aisa\Library_Assignment_Final>
```

8. Conclusion

The Mini Library Management System demonstrates thoughtful application of Python's core data structures to solve a real-world problem. The design choices reflect a balance between performance, simplicity, and maintainability while fully meeting all assignment requirements. The system provides a solid foundation that could be extended for more complex library management needs while remaining accessible and understandable for educational purposes.

The rational use of dictionaries, lists, and tuples showcases understanding of each data structure's strengths and appropriate application scenarios, resulting in an efficient and effective library management solution.

9. Reference

Chaudhri, A.B. & Zicari, R. (2020) Database Design: Know It All. Morgan Kaufmann Publishers.

Cormen, T.H., Leiserson, C.E., Rivest, R.L. & Stein, C. (2022) Introduction to Algorithms. 4th edn. MIT Press.

Lutz, M. (2023) Learning Python. 6th edn. O'Reilly Media.

Martin, R.C. (2021) Clean Architecture: A Craftsman's Guide to Software Structure and Design. Pearson Education.

McKinney, W. (2022) Python for Data Analysis. 3rd edn. O'Reilly Media.

Sedgewick, R. & Wayne, K. (2020) Algorithms. 4th edn. Addison-Wesley Professional.

Vernon, V. (2021) Implementing Domain-Driven Design. Pearson Education.

Zelle, J. (2022) Python Programming: An Introduction to Computer Science. 3rd edn. Franklin, Beedle & Associates.